

输入加速

```
// 禁止混用 cin / scanf 和 cout / print
#define IOS ios::sync_with_stdio(false);cin.tie(0);
```

输出浮点数

double 使用 printf 设定 .nlg 输出 n 位有效数字
cout 则使用 setprecision 设定输出总位数

最大公因数 GCD 欧几里得算法

```
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

最小公倍数 LCM

```
int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}
```

位运算小技巧 存在 int a, n

$a \ll n$ 等价于 $a * 2^n$ ，将 n 的二进制左移 n 位

$a \gg n$ 等价于 $\frac{a}{2^n}$ ，将 n 的二进制右移 n 位

$a \& 1$ 快速判断 a 的奇偶性，真为奇数，假为偶数

$a > 0 \&\& (a \& (a - 1)) == 0$ 快速判断 a 是否为 2 的幂（等价于 $a == 2^x$ ）

$\&$ 按位与：两个均为 1 才出 1

$|$ 按位或：有 1 出 1

注：当 a 为偶数时，使用 $a | 1$ 可以实现 $a + 1$

存在公式： $a \& b + a | b = a + b$

$\wedge$ 异或（XOR）：相同取 0，不同取 1，存在定理（ $a \wedge a = 0, a \wedge 0 = a$ ）

STL 容器小技巧 假定存在 `vector<int> a`

1. `sort(a.begin(), a.end())` 默认升序排序

基本类型可加默认降序排序器 `greater<ll>` 或自定义 lambda: `[](int a, int b){ return a>b; }`

自定义类型需要重载运算符 (`operator>`) 或自定义 lambda

当且仅当需要保持原顺序时使用 `stable_sort` (其他情况一律 `sort`)

2. 字符串 `String` 在比较和排序时按照字典序进行排序和比较。
3. `a.reverse(a.begin(), a.end())` 不包括 `a.end()` 指定的元素 (即翻转范围为 `[begin, end)`)。
4. `lower_bound` 和 `upper_bound` 使用前都必须保证容器有序。
5. `priority_queue`、`map` 和 `set` 自动维护升序 (降序需重载或使用自定义函数), 其中 `map` 和 `set` 可以去重。
6. `find()` 查找函数对于普通容器在找不到时返回右侧 (如 `a.end()`), 对于字符串返回 `string::npos`。
7. `a.erase(remove(a.begin(), a.end(), value), a.end())` 删除数组 `a` 中所有值为 `value` 的元素
`a.erase(remove_if(a.begin(), a.end(), fun), a.end())` 删除数组 `a` 中所有条件满足 `fun` 的元素
7. `vector` 的 `pop_back`、`queue` 的 `pop`、`deque` 的 `pop_back` 和 `pop_front` 都是 $O(1)$ 的时间复杂度。

二分 最大化最小值

// 目标：找到满足 check(x) 为 true 的最大 x

```
int left = 0, right = MAX_VAL;
while (left < right) {
    int mid = left + ((right - left + 1) >> 1); // 向上取整
    if (check(mid)) { // mid 满足条件，尝试找更大的，[mid, right)
        left = mid;
    } else { // mid 不满足，答案一定在左边，[left, mid-1]
        right = mid - 1;
    }
}
return left; // 或 right
```

二分 最小化最大值

// 目标：找到满足 check(x) 为 true 的最小 x

```
int left = 0, right = MAX_VAL;
while (left < right) {
    int mid = left + ((right - left) >> 1); // 向下取整
    if (check(mid)) { // mid 满足条件，尝试找更小的，[left, mid]
        right = mid;
    } else { // mid 不满足，答案一定在右边，[mid+1, right)
        left = mid + 1;
    }
}
return left; // 或 right
```

BFS 和 DFS

```
vector<vector<int>>> image;    // 邻接表
vector<bool> flag;           // 节点记录
// DFS:沿着一个方向走到头
void dfs(int i,vector<int>& res){
    res.push_back(i);        // 保存当前节点
    flag[i]=true;           // 记录已访问
    // 寻找下一个可以走的位置
    for(int& t:image[i])
        if(!flag[t])
            dfs(t,res);
}
// BFS:逐层遍历
void bfs(int i, vector<int>& res){
    queue<int> q;            //BFS 实现关键
    q.push(i);
    flag[i] = true;
    while(!q.empty()){
        int t=q.front();
        res.push_back(t);
        q.pop();
        // 根据当前节点进行扩散寻找值[子节点]
        for (int& v : image[t]) {
            if (!flag[v]) {
                q.push(v);flag[v] = true;
            }
        }
    }
}
```

并查集

```
class UnionFind {
private:
    /* 连通分量[不同集合]个数 */
    int count;           //连通分量个数
    /* 每个节点的上一节点[祖先] */
    vector<int> parent;   //祖先
    /* 以每个节点为根节点的树大小 */
    vector<int> size;     //大小
public:
    // 初始化并查集
    UnionFind(int n) : count(n), parent(n), size(n, 1) {
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }
    // 将 p 和 q 连通
    void unionSets(int p, int q) {
        int rootP = find(p); int rootQ = find(q);
        if (rootP == rootQ) return;
        // 以每个节点为根节点的树大小作为权重
        // 将小树连接到大树下面
        if (size[rootP] > size[rootQ]) {
            parent[rootQ] = rootP; size[rootP] += size[rootQ];
        } else {
            parent[rootP] = rootQ; size[rootQ] += size[rootP];
        }
        count--;
    }
    // 判断 p 和 q 是否互相连通[在同一集合内]
    bool connected(int p, int q) { return find(q) == find(p); }
    // 寻找 x 所在树的根节点
    int find(int x) {
        if (x == parent[x]) return x;
        return parent[x] = find(parent[x]);
    }
    int Count() { return count; }
};
```

一维前缀和

```
vector<int>& nums;           //原数组
int n=nums.size();
vector<int> sum(n+1,0);      //前缀和数组
//构造前缀和
for(int i=0;i<n;i++) sum[i+1]=sum[i]+nums[i];
//获取指定范围的和
auto getSum = [&](int left,int right){ return sum[right+1]-sum[left]; };
```

二维前缀和

```
vector<vector<int>>& mat;           //原数组
int n=mat.size(),m=mat[0].size();
vector<vector<int>> sum(n+1,vector<int>(m+1,0));
/* 二维数组计算前缀和 */
for(int i=1;i<=n;i++) for(int j=1;j<=m;j++) sum[i][j] = mat[i-1][j-1] + sum[i-1][j] + sum[i][j-1] - sum[i-1][j-1];
/* 获取指定矩阵和 */
auto getSum = [&](int r1,int c1,int r2,int c2){
    return sum[r2+1][c2+1] - sum[r2+1][c1] - sum[r1][c2+1] + sum[r1][c1];
};
```

二维数组转换成一维数组的变化及还原

```
vector<vector<int>> nums;           // 目标二维数组
int n=nums.size(),m=nums[0].size(); // 二维数组的行和列
vector<int> res(n*m);              // 转换后的目标一维数组
// 转换过程
for(int i=0;i<n;i++)
    for(int j=0;j<m;j++)
        res[i*m+j] = nums[i][j];
vector<vector<int>> new_nums(n,vector<int>(m)); // 还原后的二维数组
// 还原过程
for(int i=0;i<n*m;i++){
    int row = i / m, col = i % m;
    new_nums[row][col] = res[i];
}
```

三维数组转换成一维数组的变化及还原

```
vector<vector<vector<int>>> nums;           // 目标三维数组
int g=nums.size(),n=nums[0].size(),m=nums[0][0].size(); // 三维数组的数量、行和列
vector<int> res(g*n*m);                  // 转换后的目标一维数组
// 转换过程
for(int k=0;k<g;k++)
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            res[k*n*m+i*m+j] = nums[k][i][j];
// 还原后的三维数组
vector<vector<vector<int>>> new_nums(g,vector<vector<int>>(n,vector<int>(m)));
// 还原过程
for(int i=0;i<g*n*m;i++){
    int grw = i / (n*m), row = (i % (n*m)) / m, col = i % m;
    new_nums[grw][row][col] = res[i];    }
```

差分数组

```
class Difference{
    vector<int> diff;           //差分数组

public:
    /* 差分数组的初始化 */
    Difference(const vector<int>& nums){
        assert(!nums.empty()); //检查数组是否不为空
        diff.resize(nums.size());
        diff[0]=nums[0];
        for(size_t i=1;i<nums.size();i++)
            diff[i] = nums[i] - nums[i-1];
    }

    /* 差分数组的操作[val 可正可负] */
    void increment(int i,int j,int val){
        assert(i >= 0 && j < (int)diff.size() && i <= j);
        diff[i]+=val;
        //确保 j 不是最后一个元素
        //右端点为最后一个元素时不用处理
        if(j+1<(int)diff.size())
            diff[j+1]-=val;
    }

    /* 差分数组的还原 */
    vector<int> result() const {
        vector<int> res(diff.size());
        res[0]=diff[0];
        for(size_t i=1;i<diff.size();i++)
            res[i]=res[i-1]+diff[i];
        return res;
    }
}
```

拓扑排序（kanh 入度法）【有向无环图】

```
vector<vector<int>>> map(n+1);    //邻接表
vector<int> in(n+1);             //入度记录[关键]
//vector<int> list;              //拓扑排序得到的顶点序列
// 拓扑排序只能求得其中一种合法的顶点序列（满足边方向约束）
// 本题使用 DP 来求得从源点到汇点的路径数量总和
// 设 dp[i]为从所有源点走到 i 的路径数量
vector<int> dp(n+1,0);
// 建立邻接表
for(int i=0;i<m;i++){
    int u,v;cin>>u>>v;          //{出发点,到达点}
    map[u].push_back(v);         //加入邻接表
    in[v]++;                     //记录到达点的入度
}
// 拓扑排序实现的关键:队列
// 每次拿取的是入度为 0 的节点
// 即队列存取的是入度为 0 的节点
queue<int> zero;
// 寻找所有入度为 0 的点,给寻找序列建立基础
for(int i=1;i<=n;i++){
    if(in[i]==0){
        zero.push(i);
        // 该点入度为 0,即代表该点必有一条最大链
        dp[i]=1;
    }
}
while(!zero.empty()){
    // 先拿取一个入度为 0 的节点
    int u = zero.front();
    // 然后删除该节点
    zero.pop();
    // 将该节点加入序列
    //list.push_back(u);
    // 然后更新入度记录
    // 即当前由该点所连的边得到的入度全部清除
    for(int& t:map[u]){
        // 更新入度
        in[t]--;
        // 更新经由该点的路径数量
        // u 节点当前为零入度点,代表必有一条路径经由
        // 故经由该点下一步可达的点的路径中必含有经由 u 节点的路径
        // 则下一点的路径数量必是两点路径数量之和
        dp[t] = (dp[u] + dp[t]) % MOD;
        // 如果当前点入度为 0,就要将其加入队列
        if(in[t]==0) zero.push(t);
    }
}
// 所得到的 list 就是一种拓扑排序序列
int ans = 0; // 总路径数量是所有汇点（出度为 0）的路径数量之和
// 路径末点必是出度为 0 的节点
for(int i=1;i<=n;i++){
    if(map[i].empty())    ans = (dp[i] + ans) % MOD;
}
cout<<ans;
```


最小生成树（Prim 算法）

```
vector<vector<pair<int,int>>> p; //邻接表[邻居,权值]
vector<bool> flag; //访问记录
vector<vector<int>> tree; //最终生成的最小树

// 从一个点开始,每次选择连接该点权值最小的边
// 然后从这个边到下个点,依旧选择该点权值最小的、对应点未被访问的边
// 直到逐步构建出一个最小生成树
int prim(int start,int n){
    vector<bool> flag(n+1,false); //记录是否被访问过
    // 用 vector 作为底层容器,以 pair<int,int>作为数据结构的升序最小堆队列
    priority_queue<
        tuple<int,int,int>, // {权值,起点,终点}
        vector<tuple<int,int,int>>, // 底层容器
        greater<tuple<int,int,int>>
    > pq;
    int sum=0; //总和
    flag[start] = true; //标记初始点已访问
    // 存储初始点的所有边
    for(auto& edge:p[start]){
        int v=edge.first,w=edge.second;
        pq.push(make_tuple(w,start,v));
    }
    while(!pq.empty()) {
        auto t=pq.top(); // 取得连接该点中最小权值的边
        int w=get<0>(t),u=get<1>(t),v=get<2>(t);
        pq.pop(); // 去除该边
        if(flag[v]) continue; // 如果该边对应的点已访问,就跳过
        flag[v]=true; // 否则标记该点已访问
        sum += w; // 计入权值
        // 加入树[无向图要加两次]
        tree[u].push_back(v);
        tree[v].push_back(u);

        // 存入对应点的所有连接的边[扩散]
        for(auto& edge:p[v]){
            int to=edge.first,w2=edge.second;
            if(!flag[to]) pq.push(make_tuple(w2,v,to));
        }
    }
    // 有效值为[1,n],注意跳过 0
    if(find(flag.begin()+1,flag.end(),false)!=flag.end()) return -1;
    return sum;
}
```

最小生成树（Kruskal 算法）

```
struct edge{
    int x,y,w; //起点,终点,权值
    edge(int a,int b,int c):x(a),y(b),w(c) {}
};
vector<edge> E; //无向图
vector<vector<int>> tree; //最小生成树的邻接表
/* 并查集模版 */
// kruskal 算法是每次选择最小的边
// 并且检查若加入该边是否会生成环
// kruskal 通过并查集检查是否生成环
void kruskal(int n){
    // 排序使得每次拿取的都是最小的
    sort(E.begin(),E.end(),[](edge& a,edge& b){
        return a.w<b.w;
    });
    UnionFind uf(n); // 并查集防止生成的树有环
    int sum = 0,cnt = 0; // 权值总和和计数
    for(auto& e:E){
        // 合并失败就代表有连接
        if(uf.merge(e.x,e.y)){
            sum += e.w;
            tree[e.x].push_back(e.y);
            cnt++;
            if(cnt == n-1) break;
        }
    }
    // 计数不到节点数,即代表不联通
    if(cnt<n-1) cout<<"orz";
    else cout<<sum;
}
```

Dijkstra 单源最短路径【仅能在非权图中使用】

```
struct State {
    int node,dis;    // 当前节点,前往节点的距离[权值]
    State(int n, int d) : node(n), dis(d) {}
    // 小顶堆[确保升序第一个最小]
    bool operator<(const State& other) const {
        return dis > other.dis;
    }
};

vector<vector<pair<int,int>>> mp;
// Dijkstra:计算从原点开始每个点的带权距离
// 使用 BFS 的方法,即 Dijkstra 是 BFS 的加权版本
vector<int> Dijkstra(int n, int start) {
    vector<int> dis(n+1, -1);    // 从 start 到每个点的带权距离
    priority_queue<State> pq;    // 小顶堆[确保升序]
    pq.push(State(start, 0));
    while (!pq.empty()) {
        State t = pq.top();pq.pop();
        int v = t.node, w = t.dis;
        // 距离已更新过,就跳过
        if (dis[v] != -1) continue;
        dis[v] = w;
        for (pair<int,int>& p : mp[v]) {
            int y = p.first, w2 = w + p.second;
            if (dis[y] == -1) pq.push(State(y, w2));
        }
    }
    return dis;
}
```

秦九韶算法

对于多项式 $a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$

可将其分解为 $(\dots((a_n x + a_{n-1})x + a_{n-2})x + \dots + a_1)x + a_0$

证明:

$$\begin{aligned} & 5x^4 + 6x^3 + 7x^2 + 8x \\ &= (5x^3 + 6x^2 + 7x + 8)x \\ &= ((5x^2 + 6x + 7)x + 8)x \\ &= (((5x + 6)x + 7)x + 8)x \end{aligned}$$

逆元

在模 m 的条件下, 如果存在一个整数 x , 使得 $a \cdot x \equiv 1 \pmod{m}$

那么这个 x 就是 a 在模 m 下的逆元, 记作 $a^{-1} \pmod{m}$, 即 $x = a^{-1}$

同时 a 和 m 互质 (即 a 和 m 的最大公因数是 1), 互质的情况下一定能找到 x

逆元的存在就是求得模 m 的条件下, a^{-1} 的正确值

如果直接计算 a^{-1} 会在取整后得到 0, 与模运算下的除法结果不正确, 即直接算分数除法在整数模下不合法

组合数公式 $C_n^k = \frac{n!}{k! \cdot (n-k)!}$

费马小定理

如果 p 是质数, 且 a 与 p 互质 ($\gcd(a, p) = 1$), 则 $a^{p-1} \equiv 1 \pmod{p}$

那么 $a^{p-1} \equiv a \cdot a^{p-2} \equiv 1 \pmod{p}$, 即 $a^{-1} = a^{p-2} \pmod{p}$

所以对于模 p 情况下求 a^{-1} , 等价于直接计算 a^{p-2}

// 步骤 1: 预处理阶乘

```
const int MAX = 1000000;
```

```
const int MOD = 1e9 + 7; // 若题目要求需更换
```

```
vector<ll> fact(MAX + 1);
```

```
fact[0] = 1;
```

```
for (int i = 1; i <= MAX; i++) fact[i] = fact[i - 1] * i % MOD;
```

```
/* 快速幂模版 fast_pow */
```

// 步骤 2: 组合数公式

```
ll C(int n, int k) {
```

```
    if (k < 0 || k > n) return 0;
```

```
    return fact[n] * fast_pow(fact[k], MOD-2) % MOD * fast_pow(fact[n-k], MOD-2) % MOD;
```

```
}
```

扩展欧几里得定理

对于任意整数 A 和 B , 一定存在整数解 (x, y) 使得 $Ax + By = \gcd(A, B)$

求解模 p 情况下的 a^{-1} , 即求解整数 x 使得模 p 情况下 $x = a^{-1}$, 且 $a \cdot x \equiv 1 \pmod{p}$

此时根据模的定义, 还存在整数 y 使得 $ax - py = 1$

又因对于整数 a 和 p 存在 $ax + py = \gcd(a, p)$

如果 $\gcd(a, p) = 1$, 那么 $ax + py = 1$, 移项 $ax = 1 - py$

两边同取余 p 可得 $ax \equiv (1 - py) \pmod{p} \equiv (1 \pmod{p} - py \pmod{p}) \equiv 1 \pmod{p}$

此时求解扩展欧几里得中的整数 x , 即可求得模 p 情况下的 a^{-1}

// 求出 $ax + by = \gcd(a, b)$

```
ll exgcd(ll a, ll b, ll& x, ll& y) {
```

```
    if (b == 0) { x = 1, y = 0; return a; }
```

```
    ll g = exgcd(b, a % b, y, x);
```

```
    y -= a / b * x; return g;
```

```
}
```

// 求逆元(即得到求余情况下的 a^{-1})

```
ll inv(ll a, ll mod) {
```

```
    ll x, y;
```

```
    ll g = exgcd(a, mod, x, y);
```

```
    if (g != 1) return -1;
```

```
    return (x % mod + mod) % mod;
```

```
} /* 后面通过公式求解组合数即可 */
```

线段树 区间全部修改为 k

```
const ll NO_TAST = -1e18;    // 特殊值,用于退出程序
// 懒标记任务函数
// 当前节点编号 节点代表的区间[左边界, 右边界]
void push_down(int p, int l, int r) {
    // 0 代表没有任务
    if (lazy[p] == NO_TAST) return;
    if (l == r) { tree[p] += lazy[p]; lazy[p] = 0; return; }
    int mid = l + ((r - l) >> 1);
    // 更新左右节点
    tree[p << 1] = lazy[p] * (mid - l + 1);
    tree[p << 1 | 1] = lazy[p] * (r - mid);
    // 懒标记任务传递
    lazy[p << 1] = lazy[p];
    lazy[p << 1 | 1] = lazy[p];
    // 标记已结束任务
    lazy[p] = 0;
}
// 单点更新 ans[i] = x
// 节点区间左边界和右边界 节点编号 原数组下标 目标修改值
void _update(int l, int r, int p, int x, int y, ll k) {
    // 叶子节点直接修改
    if (x <= l && r <= y) {
        tree[p] = k * (r - l + 1);
        lazy[p] = k;
        return;
    }
    push_down(p, l, r); // 标记懒标记
    int mid = l + ((r - l) >> 1);
    if (x <= mid) _update(l, mid, p << 1, x, y, k);
    if (y > mid) _update(mid + 1, r, p << 1 | 1, x, y, k);
    tree[p] = tree[p << 1] + tree[p << 1 | 1];
}
```

埃氏筛 找出小于或等于 n 的素数

```
vector<bool> prime(n+1, true);
for (int i = 2; i*i <= n; i++) {
    if (prime[i]) // 当前 p 为素数
        for (int p = i*i; p <= n; p++)
            prime[i] = false;
    // 则 p 的所有倍数均非素数
}
```

快速幂 计算 a^b ，pow(a, b)的优化版

```
const int MOD = 1e9 + 7;
long long fast_pow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}
```

数学知识

1. 等差数列的首项为 a_1 ，公差为 n ，则其通项公式为 $a_n = a_1 + (n - 1) * d$ ，
求和公式为 $S_n = n * a_1 + \frac{n * (n-1)}{2} * d$ 。
2. 等比数列的首项为 a_1 ，公比为 q ，则其通项公式为 $a_n = a_1 * q^{n-1}$ ，
求和公式为 $S_n = a_1 * \frac{1-r^n}{1-r}$ 。
3. 素数等于质数，要求均为因子只有 1 和他本身。
4. 如果 t 为 MEX（集合中没有出现的最小自然数），则集合中必有 $\{0, \dots, t-1\}$ 的完整序列并且不存在 t。
5. 设三点为 A (x_1, y_1) ，B (x_2, y_2) ，C (x_3, y_3) 。
如果三点共线，必满足 $(x_1 - x_2) * (y_3 - y_1) - (y_2 - y_1) * (x_3 - x_1) = 0$ 。
三点所在三角形面积： $S = \frac{1}{2} * abs((x_2 - x_1) * (y_3 - y_1) - (y_2 - y_1) * (x_3 - x_1))$ 。
6. 拉格朗日四平方和定理：任何一个正整数，都可以写成至多 4 个平方数的和。
7. 勒让德三平方和定理：当 $n \neq 4^a(8b + 7)$ ，其中 $a, b \in Z$ ，n 就可以表示为三个平方数之和。

数据范围 N	预估时间复杂度	可能的算法	核心思想
$N \leq 10 \sim 12$	$O(N!)$	阶乘级暴力 / 全排列	枚举所有排列组合。
$N \leq 18 \sim 22$	$O(2^N * N)$ 或 $O(2^N * N^2)$	状压 DP (状态压缩动态规划)	用一个二进制整数来表示“集合”状态。
$N \leq 30 \sim 50$	$O(2^{(N/2)})$	折半搜索 (Meet-in-the-Middle)	将问题一分为二，分别暴力，再合并结果。
$N \leq 100$	$O(N^4)$	四重循环暴力 / 某些 DP	简单粗暴的枚举。
$N \leq 400 \sim 500$	$O(N^3)$	Floyd-Warshall 算法 / 区间 DP	所有点对最短路；较复杂的 DP 模型。
$N \leq 2000 \sim 5000$	$O(N^2)$	基础 DP / 图论遍历 / 暴力枚举	大部分二维 DP、朴素的最短路/MST 算法。
$N \leq 10^5$	$O(N * \sqrt{N})$	分块算法 / 莫队算法	将数据分块处理的根号复杂度算法。
$N \leq 10^5 \sim 10^6$	$O(N \log N)$	排序 / 堆 / 二分 / 分治 / 线段树 / 树状数组 / 快速幂 / Dijkstra(优先队列优化)	这是最重要的区间！大部分“中档题”和“难题”都落在这里。看到这个范围，就要立刻想到这些算法。
$N \leq 10^6 \sim 10^7$	$O(N)$	贪心 / 双指针(滑动窗口) / 并查集 / BFS/DFS / KMP / 线性筛 / DP(滚动数组优化)	线性时间复杂度的算法。要求算法必须极其高效。
$N \leq 10^9$	$O(\sqrt{N})$	数论（质因数分解等）	$O(N)$ 都会超时，必须用根号级别的数学技巧。
$N \leq 10^{18}$	$O(\log N)$	快速幂 / 数学（矩阵快速幂、数位 DP）	巨大的输入，暗示解法与输入的“大小”无关，而与其“位数”或“二进制表示”有关。
多次查询 Q	$O(Q * \log N)$ 或 $O(Q)$	数据结构 / 预处理	Q 很大时，单次查询必须高效。通常需要线段树、树状数组、ST 表等，或者通过预处理（如前缀和）来达到 $O(1)$ 查询。